

No Safe Circle

Assignment #10 - Complete AI Development Pipeline

Vincent J. Liguori

Playable Game	https://cathode26.github.io/NoSafeCircle/
Pipeline Video	https://cathode26.github.io/NoSafeCircle/video.html
GitHub Repository	https://github.com/cathode26/NoSafeCircle

About the project. No Safe Circle is a fixed-isometric action dungeon game made in Unity. The player controls a wizard with the mouse, moves through dungeon spaces, fights enemies, uses health and mana, and interacts with doors and the environment. The project also became a test bed for an AI-assisted development pipeline that plans work, routes it between AI providers, generates Unity code and tests, records human validation, and preserves evidence showing exactly what was delivered.

What I Built Around the Game

GDD + Design Reconciliation

The GDD starts as the game-design source, but it is not frozen. Reconciliation compares the design with the existing Unity project and proposed work. Repeated passes expose missing ownership or requirements; if the design is genuinely ambiguous, I make the decision, update the canonical GDD, and run the analysis again.

GDD RAG

A deterministic retrieval system indexes the current GDD so reviewers can use focused design context instead of always sending the entire document.

TaskGraph

A persistent graph of game-development tasks with requirements, parent/child relationships, dependencies, Unity ownership, validation expectations, and delivery evidence.

Task Decomposition

Large features can be broken into smaller AI-sized jobs. OpenAI/Codex and Claude can alternate author/reviewer roles so the latest author does not approve its own decomposition.

AgentRuntime / TaskExecution

A provider-neutral runtime lets the pipeline use both Claude Code and OpenAI/Codex. This supports competing model families, independent review, provider choice by job, and resilience when one provider has usage or token limits.

Software Architect

A supervised polling architect evaluates available work, dependencies, active branches/checkouts, and likely Unity integration conflicts. It can recommend START, WAIT, or HUMAN REVIEW and can recommend provider/reasoning routing while deterministic code retains repository authority.

GitHub Issue Workflow

GitHub Issues are the durable operational record for task ownership, phase, branch, commit, checkout, human handoff, and PASS/FAIL results. GitHub Actions validate the handoff transitions.

Game Task Agent + ExecutionCrew

One real task is isolated in its own checkout/branch, scoped, implemented, tested, independently validated, repaired if necessary, patch-checked, committed, and pushed before the human Unity handoff.

Unity Validation + TaskDelivery

Unity Edit Mode/Play Mode results and human runtime validation are tied to exact commits. TaskDelivery packages the files, tests, logs, commit identity, and human evidence used to prove the task.

Historical Context Sessions

CURRENT_CONTEXT.md and focused session handoffs let new ChatGPT, Claude, or Codex sessions continue a large project without reconstructing the architecture from memory. Live Git, TaskGraph, tests, and GitHub state are re-verified before mutation.

ArchitectureReview

Eight specialist reviewers, a synthesis pass, and a final adversarial critic challenge the pipeline architecture, overengineering, GDD iteration strategy, human intervention, and whether the system is actually improving game-development throughput.

How Work Moves Through the Pipeline

The important part is that the system is not one long prompt. Each stage has a narrow job, and the pipeline deliberately stops when a decision belongs to me instead of an AI.

[HUMAN] Write and approve the initial GDD.

[RECONCILIATION] Compare the GDD with the current Unity project and proposed work.

[HUMAN WHEN NEEDED] Clarify a real design ambiguity, update the GDD, and run reconciliation again.

[TASKGRAPH] Store the approved work as structured tasks, dependencies, ownership, and validation requirements.

[TASK DECOMPOSITION] Break down work that is too large; Claude and OpenAI/Codex can independently review each other.

[SOFTWARE ARCHITECT] Choose safe work, avoid likely integration conflicts, and recommend execution routing.

[GAME TASK AGENT] Claim the task and create/resume its isolated Git checkout and task branch.

[AI SUPERVISOR] Inspect the repository and propose the exact implementation/test file scope.

[EXECUTIONCREW] Implement the Unity feature, create/update tests, validate the result, and perform a bounded repair cycle when needed.

[PIPELINE] Verify the actual Git patch and apply the reviewed changes to the canonical task branch.

[UNITY TESTING] Run the required automated Unity tests against the exact commit.

[HUMAN WHEN REQUIRED] Open Unity, follow the task Issue checklist, and post PASS or FAIL for visual/interactive behavior.

[FAIL] Return the durable task to repair.

[PASS] Move to delivery evidence.

[TASKDELIVERY] Bind requirements to exact files, commit identity, automated tests, and human validation.

[HUMAN] Review truth-bearing evidence claims and retain final merge authority.

[BUILD/PUBLISH] Build the Unity Web version and publish the playable build.

Why the GDD and TaskGraph Were Iterative

Building the TaskGraph acted like a design pressure test. Statements that were clear enough to a human game developer became ambiguous when separate AI workers needed exact ownership boundaries. The process exposed questions about player health, enemy health/defeat, enemy registration, doorway crossing, navigation, restart state, input, and other shared systems. Each clarification made the GDD more explicit, and then the graph was reconciled again.

This means the cost of one TaskGraph/reconciliation pass is not the cost of creating the TaskGraph. The initial graph was refined through a dozen or more reconciliation/review iterations. A practical cost estimate has to account for repeated passes, not just one model call.

Human Workflow Through GitHub Issues

1. I start the supervised Software Architect session.
2. When a task is assigned, its GitHub Issue becomes the durable running record: state, owner, phase, branch, commit, checkout, and next action.
3. Normally I do nothing while the AI is coding. The Game Task Agent commits and pushes the task before asking me to test it.
4. When Unity runtime validation is required, the task Issue contains the exact commit, checkout, scene/setup, numbered Play Mode steps, expected result, and PASS/FAIL format.
5. The NSC-Vincent Issue acts as my notification inbox and points me to the real task Issue that needs attention.
6. I test the exact commit in Unity and post PASS or FAIL on the task Issue.
7. GitHub Actions validate the response. FAIL routes to repair; PASS routes to delivery evidence.
8. I review final evidence claims that require judgment, and I retain final merge authority.

Measured Cost Reduction

I ran a controlled reviewer replay using the same candidate and the same Claude model. One arm received the full GDD; the other used smaller GDD-RAG context. Both returned PASS with no findings.

Metric	Full GDD	GDD RAG	Change
Input tokens	392,429	233,319	40.5% fewer
Output tokens	10,730	10,490	2.2% fewer
Total tokens	403,159	243,809	39.5% fewer
Cost	\$0.6327	\$0.5235	17.3% cheaper
Review result	PASS / no findings	PASS / no findings	Same result

The important point is repetition: reducing one review by 39.5% total tokens matters more when similar reconciliation/review work is performed many times while the GDD and TaskGraph evolve.

Examples of Pipeline-Delivered Game Work

Enemy Health / Defeat: Damage, defeat, reset behavior, and active-enemy integration; authoritative Play Mode validation included a 15/15 passing test set.

Encounter Admission / Enemy Limit: Keeps active enemies within the allowed cap while preserving pursuer priority and pending work; automated Play Mode coverage verified the admission behavior.

Isometric Sprite Sorting: World-space SpriteRenderer foundation for the wizard, doors, walls, and other world objects; automated tests plus human front/behind runtime validation.

Permanent Video Link

The PDF always points to <https://cathode26.github.io/NoSafeCircle/video.html>. The current video.html page contains a temporary portfolio video. When the Assignment 10 walkthrough is ready, I can replace the embedded video on that page without changing this PDF.